Cmod A7-35T RISC-V MCU

Datasheet & Reference Manual

MicroBlaze-V soft MCU on the Digilent Cmod A7-35T — peripherals, pins, power, and boot guides

Platform	Xilinx Artix-7 (xc7a35tcpg236-1) — Cmod A7-35T
Processor	MicroBlaze RISC-V
System Clock	100 MHz (12 MHz on-board oscillator × PLL)
Toolchain	Vivado & Vitis 2025.2

Document DS-CMOD7-0001 · Revision A · July 2026

Table of Contents

1 Device Overview

1.1 Introduction

The Cmod A7-35T RISC-V MCU is a soft microcontroller implemented on the Artix-7 FPGA of the Digilent Cmod A7-35T module. To the programmer it behaves like a commercial flash-based MCU: firmware is stored in the on-board QSPI flash and boots automatically in under a second at power-on, and the full AMD Vitis/JTAG development flow is available on the same board over a single USB cable, with no jumpers or mode switching.

The system is built around a MicroBlaze-V (RISC-V) processor with a three-level memory hierarchy — tightly-coupled BRAM (ITCM/DTCM), cached external SRAM, and QSPI flash storage — and a class-based AXI peripheral address map.

1.2 Features

- **CPU** — 32-bit RISC-V (RV32IM + bit-manipulation) at 100 MHz; 16 KB I-cache + 16 KB D-cache (write-through, 32-byte lines)
- **Memory** — 128 KB block RAM (32 KB bootloader + 32 KB ITCM + 64 KB DTCM, single-cycle); 512 KB SRAM for application code and data (cached); 4 MB QSPI flash (bitstream + application storage)
- **GPIO** — 28 bidirectional pins in four 7-bit groups, plus 4 dedicated external interrupt inputs
- **Timers** — 3 × 32-bit general-purpose timers with interrupts; 3 dedicated PWM output channels
- **Communication** — 2 × 16550 UART (USB + DIP header); I2C master (100 kHz, 50 ns glitch filter); SPI master (software-settable clock ≈1.6 – 25 MHz, 2 slave selects)
- **Analog** — 2 external ADC inputs (12-bit XADC, 0–3.3 V range) plus on-die temperature and supply monitors
- **I/O** — 48-pin DIP form factor, 3.3 V LVCMOS (not 5 V tolerant)
- **Boot** — standalone boot from flash (< 1 s) via the on-chip bootloader, or JTAG load/debug from Vitis; both coexist without reconfiguration
- **Debug** — JTAG breakpoints, single-step, register and memory inspection

1.3 System Block Diagram

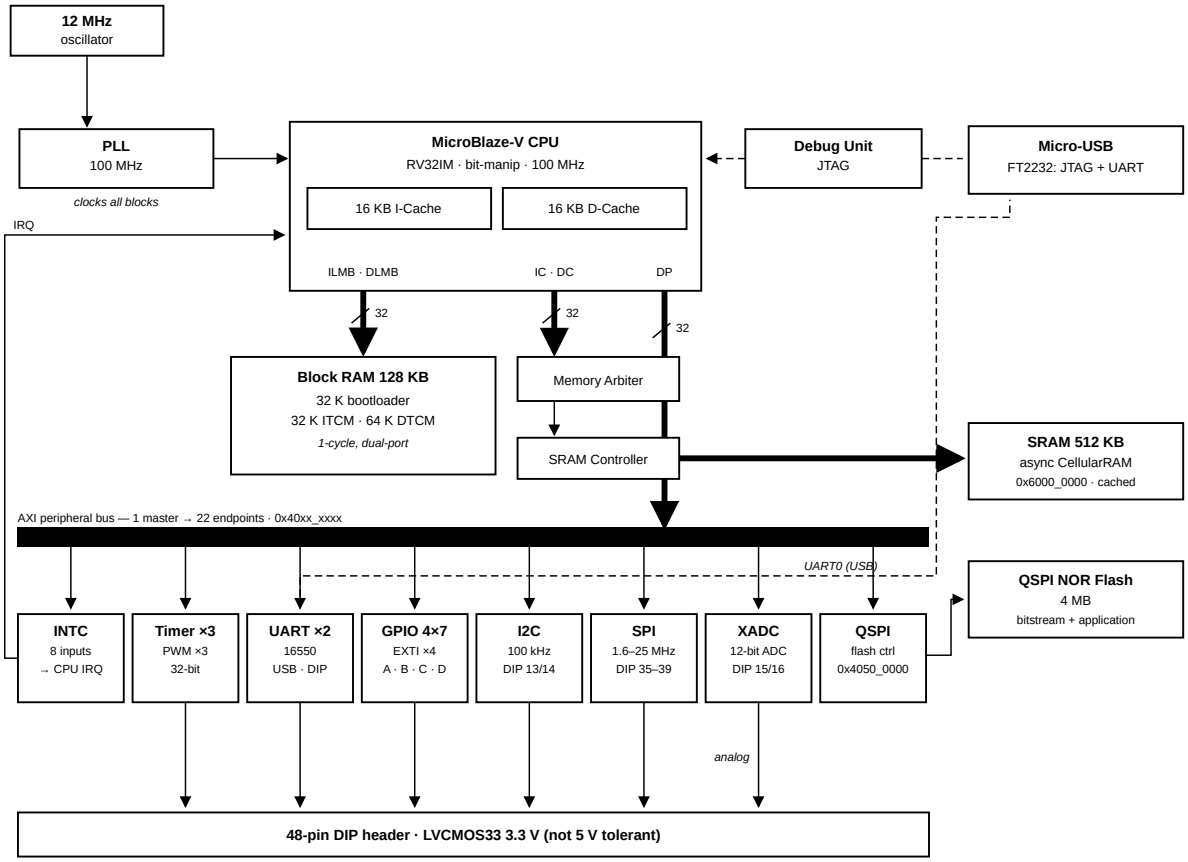

Figure 1: System Architecture

1.4 Reference Documents

- Digilent *Cmod A7 Reference Manual* — board schematics, connectors, and the power tree
- AMD/Xilinx *DS181, Artix-7 FPGAs Data Sheet* — absolute maximum ratings and DC/AC characteristics behind Chapter 5

- AMD/Xilinx LogiCORE IP product guides (PG series) — full register maps for the AXI peripherals (e.g. PG153 for the Quad SPI controller)
- The per-topic markdown under `docs/` in the course repository — the single source this datasheet is built from

2 System Architecture

2.1 MicroBlaze RISC-V Core

Item	Value
Architecture	32-bit RISC-V — RV32IM + bit-manipulation (hardware multiply/divide)
Clock	100 MHz
Buses	LMB to local BRAM (1-cycle) · AXI to peripherals and external memory
Cache	16 KB I-Cache + 16 KB D-Cache, write-through, 32 B lines — covers exactly the 512 KB SRAM (`0x6000_0000`–`0x6007_FFFF`)
Debug	JTAG: breakpoints, register inspection, memory read/write

Description: The main processor core. Executes user firmware and reaches every peripheral through the AXI interconnect. Only the SRAM range is cached — BRAM is already single-cycle, and peripheral registers must never be cached.

2.2 Bus Architecture

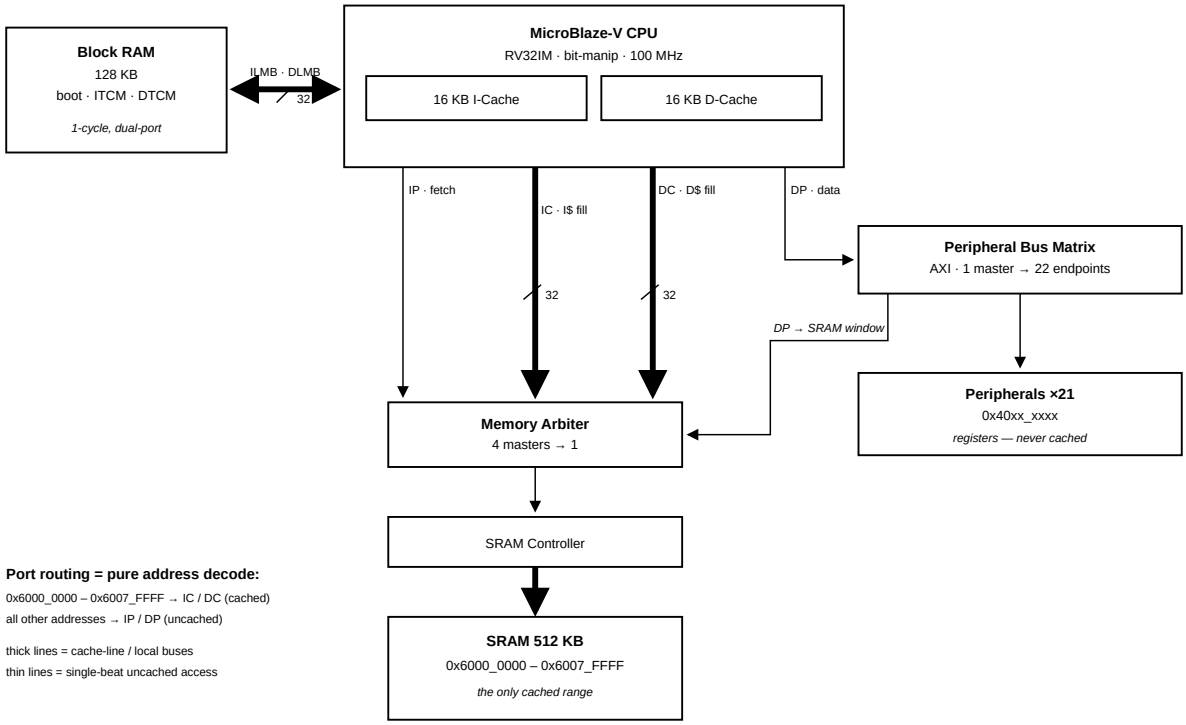

Figure 2: Bus & Cache Topology

Description: AXI crossbar that fans the processor's data port out to all peripheral endpoints. Routing is pure address decoding — the diagram above shows which path each address range takes (a second, smaller interconnect merges the cache and debug masters in front of the SRAM controller).

2.3 Interrupt Controller

Item	Value
Base Address	`0x4040_0000`
Interrupt Sources	8

Interrupt Mapping:

Channel	Source	Description
In0	`timer_0`	System timer 0 interrupt
In1	`timer_1`	System timer 1 interrupt
In2	`timer_2`	System timer 2 interrupt

Channel	Source	Description
In3	`uart_1`	External UART interrupt
In4	`uart_USB`	USB UART interrupt
In5	`INT_0_3`	External GPIO interrupt (4-bit)
In6	`i2c_0`	I2C controller interrupt
In7	`spi_0`	External SPI master interrupt

2.4 Clocking

Description: Multiplies the 12 MHz on-board oscillator up to the 100 MHz system clock with an MMCM/PLL. Its `locked` signal indicates clock stability and gates the release of system reset.

2.5 Reset

Description: Generates synchronized reset signals for the CPU, bus fabric and peripherals, releasing them only after the clock is stable. External reset source: on-board button BTN0 (A18, active-high).

2.6 Debug Module

Description: MicroBlaze Debug Module providing JTAG debug access for breakpoints, register inspection, and memory read/write. It can also trigger a system reset from the debugger (used by `xsdb` / Vitis debug sessions).

2.7 Complete Address Map

Peripheral addresses follow a class-based convention — **`0x40[C]x_xxxx`, where `C` is the peripheral class** (1 MB per class, 64 KB per instance). Reading an address immediately tells you what kind of device it is: class 0 = GPIO, 1 = Timer, 2 = PWM, 3 = UART, 4 = INTC, 5 = QSPI control, 6 = XADC, 7 = I2C, 8 = SPI, 9 = SPI clock control.

Base Address	Range	Peripheral	Type	Category
`0x0000_0000`	128K / 128K	Local Memory (BRAM)	BRAM	Memory
`0x4000_0000`	64K	board_led_2bits	axi_gpio	GPIO
`0x4001_0000`	64K	board_button	axi_gpio	GPIO
`0x4002_0000`	64K	board_rgb	axi_gpio	GPIO
`0x4003_0000`	64K	gpio_A_0_6	axi_gpio	GPIO
`0x4004_0000`	64K	gpio_B_0_6	axi_gpio	GPIO
`0x4005_0000`	64K	gpio_C_0_6	axi_gpio	GPIO
`0x4006_0000`	64K	gpio_D_0_6	axi_gpio	GPIO
`0x4007_0000`	64K	INT_0_3	axi_gpio	Interrupt
`0x4010_0000`	64K	timer_0	axi_timer	Timer
`0x4011_0000`	64K	timer_1	axi_timer	Timer
`0x4012_0000`	64K	timer_2	axi_timer	Timer
`0x4020_0000`	64K	PWM_0	axi_timer	PWM
`0x4021_0000`	64K	PWM_1	axi_timer	PWM
`0x4022_0000`	64K	PWM_2	axi_timer	PWM
`0x4030_0000`	64K	uart_USB	axi_uart16550	Communication
`0x4031_0000`	64K	uart_1	axi_uart16550	Communication
`0x4040_0000`	64K	axi_intc	axi_intc	System
`0x4050_0000`	64K	axi_quad_spi_0 (QSPI flash controller)	axi_quad_spi	Memory
`0x4060_0000`	64K	xadc_wiz_0	xadc_wiz	ADC
`0x4070_0000`	64K	i2c_0 (DIP 13/14)	axi_iic	Communication
`0x4080_0000`	64K	spi_0 (external master, DIP 35–39)	axi_quad_spi	Communication
`0x4090_0000`	64K	spi_0_clk (SPI clock control)	clock control	Communication
`0x6000_0000`	512K	axi_emc_0 (exact physical fit, I/D-cached)	axi_emc	Memory

3 Memory Hierarchy

3.1 Memory Hierarchy Overview

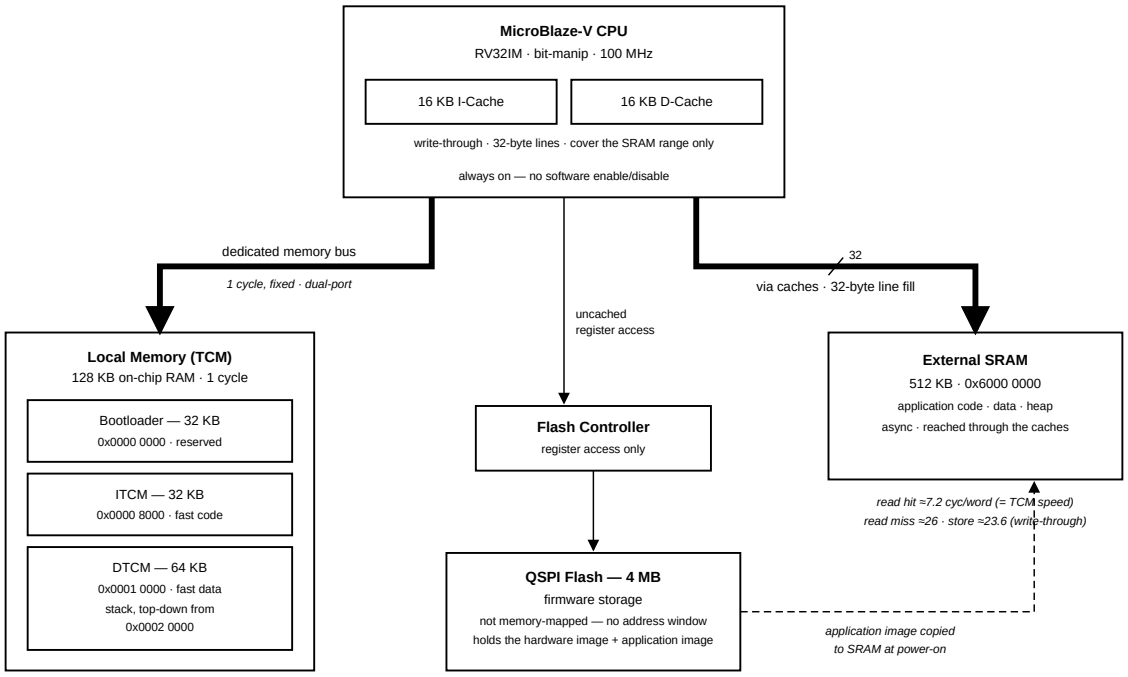

Figure 3: Memory Hierarchy

Level	Memory	Size	Access	Role
1	ITCM + DTCM (tightly-coupled RAM)	32 KB + 64 KB	1 cycle, fixed	interrupt handlers, stack, hot data
2	I-Cache + D-Cache	16 KB + 16 KB	adds ≈0 cycles on hit	transparent acceleration of SRAM
3	External SRAM	512 KB	≈26 cycles/word on miss	application code, data, heap
—	QSPI flash	4 MB	not directly addressable	firmware storage, read at boot

The tightly-coupled memories sit on a dedicated dual-port memory bus with guaranteed single-cycle access; they are never cached because they are already as fast as a cache hit. The external SRAM is the application's main memory; it is reached through the caches, so hot code and data run at tightly-coupled speed while the total space is 512 KB. The QSPI flash has no CPU address window: firmware is stored in flash and executed from SRAM, and the bootloader performs the copy at every power-on.

3.2 Memory Map

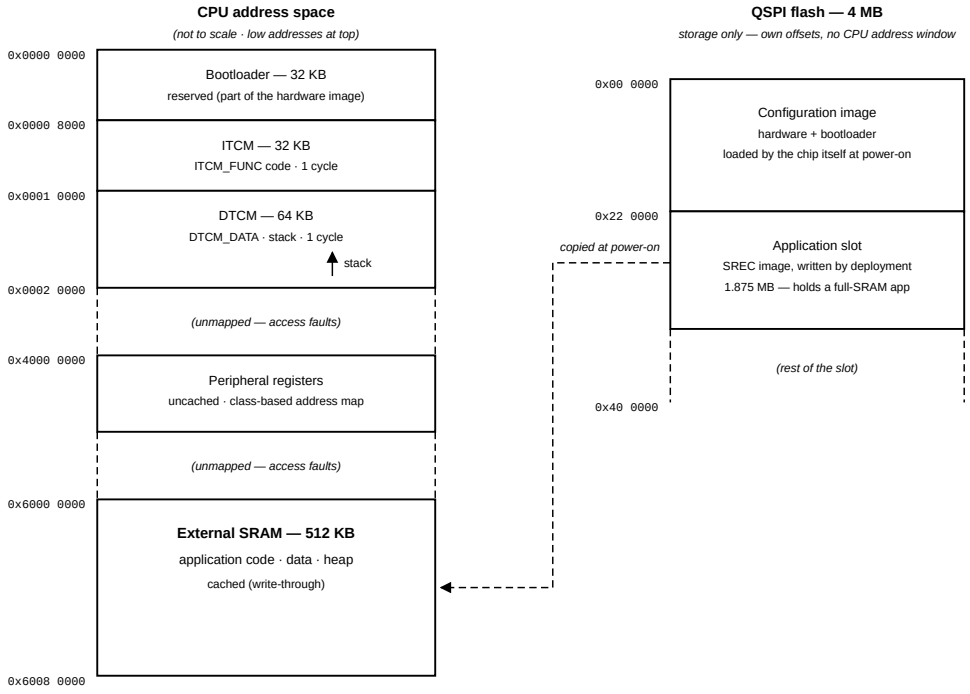

Figure 4: Memory Map

Address range	Size	Region	Access
`0x0000_0000` – `0x0000_7FFF`	32 KB	Bootloader (reserved)	1 cycle
`0x0000_8000` – `0x0000_FFFF`	32 KB	ITCM — application fast code	1 cycle
`0x0001_0000` – `0x0001_FFFF`	64 KB	DTCM — fast data; stack grows down from `0x0002_0000`	1 cycle
`0x4000_0000` – `0x40FF_FFFF`	—	Peripheral registers	uncached
`0x6000_0000` – `0x6007_FFFF`	512 KB	External SRAM — application code / data / heap	cached

Address decoding is exact: an access outside these ranges raises a bus error instead of aliasing onto real memory, so a stray pointer is detected rather than silently corrupting data. The QSPI flash does not appear in this map; it is accessed only through the flash controller's registers. Peripheral registers and base addresses are listed in Section 2.7 and detailed in Section 5.

3.3 Local Memory (ITCM / DTCM / Bootloader)

Item	Value
Address	`0x0000_0000` – `0x0001_FFFF` (128 KB)
Access	1 cycle, dual-port — instruction and data sides read simultaneously
Layout	32 KB bootloader + 32 KB ITCM + 64 KB DTCM

Description: Instruction and data local memory implemented with FPGA Block RAM — functionally this MCU's tightly-coupled memory (TCM): the ILMB/DLMB ports play the same role as ITCM/DTCM on other MCU cores (e.g. Cortex-M7), giving 1-cycle access outside the cache path. Layout: `0x0000`–`0x7FFF` (32 KB) holds the bootloader (restored from flash at every configuration); `0x8000`–`0xFFFF` (32 KB) is the application's ITCM for interrupt handlers and timing-critical code (`ITCM_FUNC`, copied out of the SRAM image by `mcu_init()` at startup); `0x10000`–`0x1FFFF` (64 KB) is the DTCM, holding the stack (top-down from `0x20000`) and `DTCM_DATA` fast data.

3.4 External SRAM (Cellular RAM)

Item	Value
Base Address	`0x6000_0000`

Item	Value
Size	512 KB — `0x6000_0000` – `0x6007_FFFF`, an exact physical fit
Memory	On-board asynchronous SRAM (Cellular RAM)
Cache	Fully covered by the I-Cache and D-Cache

Description: External memory controller for the on-board 512 KB SRAM — the main application memory: standalone boot copies the program here and executes it. Raw access latency is higher than Block RAM, but with both caches covering this range, hot code and data run near BRAM speed.

3.5 QSPI Flash

Item	Value
Base Address	`0x4050_0000`
Flash Device	On-board 4 MB QSPI NOR flash (Macronix; Micron on older boards)
Mode	CPU-programmable register mode — flash contents are **not** memory-mapped
FIFO	256 B TX/RX — one full flash page (256 B) per transfer

Description: Controls the on-board Quad-SPI NOR Flash. The full register set (control, status, TX/RX FIFO, slave select) is exposed at `0x4050_0000`, so the CPU can issue any SPI command — read (`0x0B`), Write Enable (`0x06`), Sector/Block Erase (`0x20`/`0xD8`), Page Program (`0x02`), status poll (`0x05`). This keeps the flash fully CPU-programmable at runtime (the Vitis `XSpi` driver wraps the register protocol).

Flash contents are **not memory-mapped**: there is no XIP window, and code cannot execute from flash directly. The standalone-boot design instead copies the application from flash into SRAM at power-on (see Section 7.2).

> **Design note:** a read-only memory-mapped XIP window and CPU-programmable register mode are mutually exclusive in this controller. This project chooses register mode: keeping the flash CPU-programmable at runtime was judged more valuable for the course than execute-in-place, and the 512 KB SRAM + I-cache serves execution instead.

3.6 Caches

Item	Value
Instruction cache	16 KB, 32-byte lines
Data cache	16 KB, 32-byte lines
Write policy	write-through — every store is forwarded to SRAM
Cached range	exactly the SRAM: `0x6000_0000` – `0x6007_FFFF`
Software control	none — always on for the cached range

Only the SRAM range is cached. The tightly-coupled memories are already single-cycle, and peripheral registers must observe every access, so neither is ever cached. There is no cache enable/disable or flush to manage in normal application code.

The write-through policy has two practical consequences:

- Reads benefit fully from the cache: a hit costs the same as a tightly-coupled RAM access.
- Writes always pay the SRAM latency, even on a cache hit. Frequently written data (counters, buffers filled in interrupt handlers) therefore belongs in the DTCM.

3.7 Measured Access Latencies

Measured on the board at 100 MHz with an optimization-enabled word-copy loop. Values are cycles per 32-bit word and include the loop's own ≈7-cycle overhead; the difference between rows is the memory's contribution.

Access	Cycles/word	Notes
Tightly-coupled RAM read	≈7.2	equals the bare loop; the memory adds no cycles
SRAM read, cache **hit**	≈7.2	a hit is as fast as tightly-coupled RAM
SRAM read, cache **miss**	≈26	≈151 cycles to fill one 32-byte line
SRAM write (write-through)	≈23.6	every store pays the SRAM latency

> **Note:** Measure with compiler optimization enabled. An unoptimized build inflates the loop overhead to the point where cached and uncached accesses look almost identical; the memory effects are hidden by the extra instructions.

3.8 Memory Placement Guidance

What	Put it in	How (course template)
Interrupt handlers, timing-critical loops	ITCM	tag the function `ITCM_FUNC`
Frequently-written data, DMA-like buffers	DTCM	tag the variable `DTCM_DATA`
Stack	DTCM	placed there by the template's linker script
Everything else — code, constants, data, heap	SRAM	default
Firmware image	QSPI flash	written by the deployment step

As a general rule, place code and data in SRAM by default and move them to the TCMs when timing must be predictable rather than merely fast. A function in SRAM usually runs at cache speed, but its worst case (a cold cache line) costs three to four times more; for an interrupt handler this spread appears directly as latency jitter. In the tightly-coupled memories the worst case equals the best case.

At power-on the bootloader copies the application image from flash into SRAM (see Section 7.2); the template's `mcu_init()` then copies `ITCM_FUNC` code out of the SRAM image into the ITCM and zeroes the DTCM. For this reason the template requires `mcu_init();` to remain the first line of `main()`: nothing placed in the tightly-coupled memories works before it runs. The memory tour printed at boot lists each region's live addresses and serves as a check that the layout is in effect.

4 Pinout

4.1 DIP Connector Overview

The Cmod A7-35T has a 48-pin DIP connector (J1). All digital I/O pins operate at **LVCMOS33** (3.3 V logic level) with **no series resistors**.

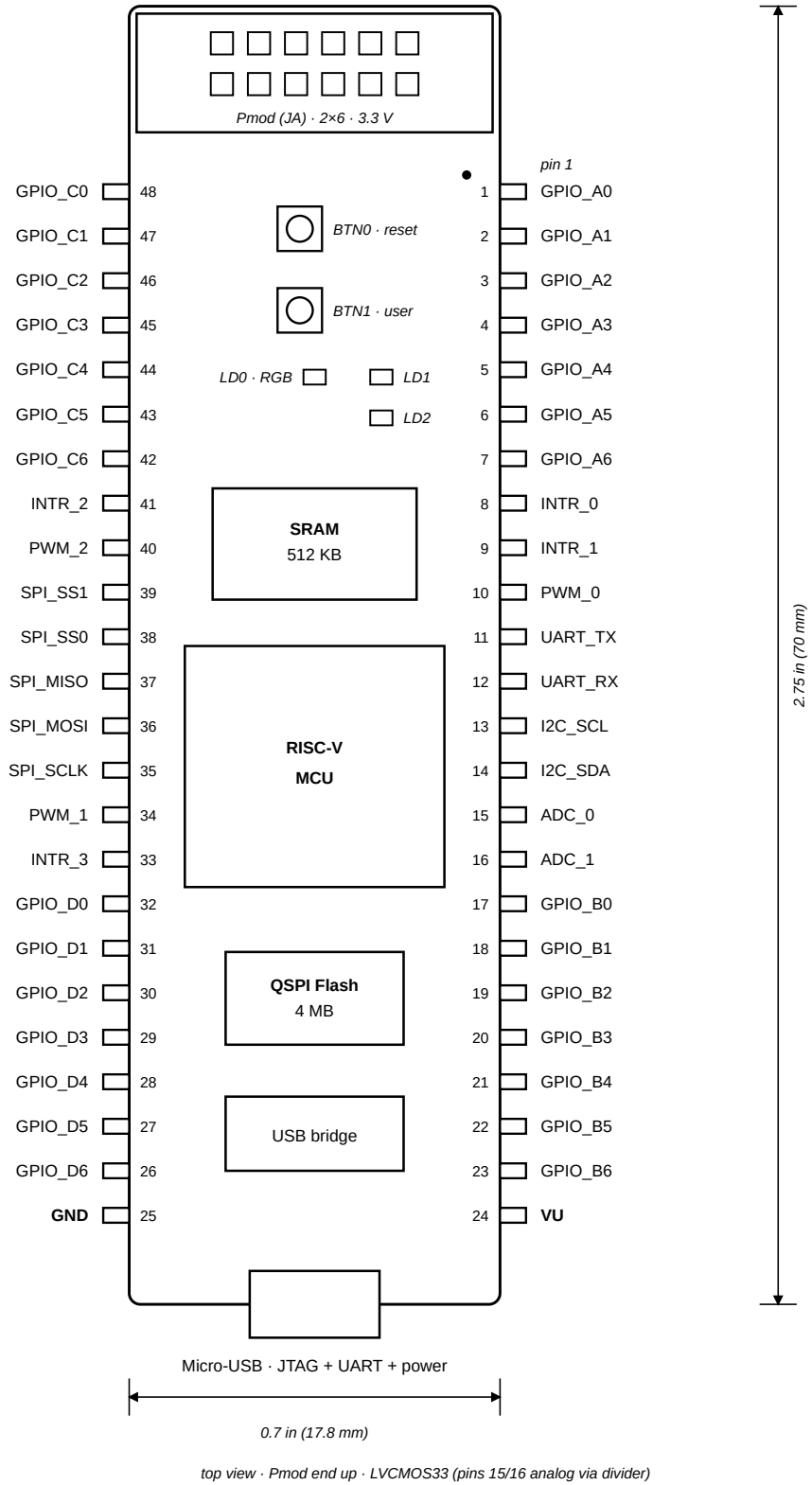

Figure 5: Cmod A7-35T DIP pinout (top view)

Category	Count
GPIO (4 groups × 7-bit)	28
External Interrupts	4
PWM Outputs	3
UART 1 (TX + RX)	2
I2C (SCL + SDA)	2
SPI (SCLK/MOSI/MISO/SS×2)	5
Analog Inputs (XADC)	2
Power (VU + GND)	2
Total	**48**

Peripheral registers and base addresses are listed in Section 2.7 and detailed in Section 5.

4.2 Pin Map — Left Side (Pin 1–24)

DIP Pin	Signal Name	FPGA Pin	Direction	Function
1	GPIO_A0	M3	I/O	General purpose GPIO, Group A bit 0
2	GPIO_A1	L3	I/O	General purpose GPIO, Group A bit 1
3	GPIO_A2	A16	I/O	General purpose GPIO, Group A bit 2
4	GPIO_A3	K3	I/O	General purpose GPIO, Group A bit 3
5	GPIO_A4	C15	I/O	General purpose GPIO, Group A bit 4
6	GPIO_A5	H1	I/O	General purpose GPIO, Group A bit 5
7	GPIO_A6	A15	I/O	General purpose GPIO, Group A bit 6
8	INTR_0	B15	Input	External interrupt input 0
9	INTR_1	A14	Input	External interrupt input 1
10	PWM_0	J3	Output	PWM output channel 0
11	UART_TX	J1	Output	UART 1 transmit
12	UART_RX	K2	Input	UART 1 receive
13	I2C_SCL	L1	I/O (open-drain)	I2C clock (external 4.7 kΩ pull-up recommended)
14	I2C_SDA	L2	I/O (open-drain)	I2C data (external 4.7 kΩ pull-up recommended)
15	ADC_0	G3 / G2	Analog	XADC VAUX4 (ain_p[15] / ain_n[15])
16	ADC_1	H2 / J2	Analog	XADC VAUX12 (ain_p[16] / ain_n[16])
17	GPIO_B0	M1	I/O	General purpose GPIO, Group B bit 0
18	GPIO_B1	N3	I/O	General purpose GPIO, Group B bit 1
19	GPIO_B2	P3	I/O	General purpose GPIO, Group B bit 2
20	GPIO_B3	M2	I/O	General purpose GPIO, Group B bit 3
21	GPIO_B4	N1	I/O	General purpose GPIO, Group B bit 4
22	GPIO_B5	N2	I/O	General purpose GPIO, Group B bit 5
23	GPIO_B6	P1	I/O	General purpose GPIO, Group B bit 6
24	**VU**	—	Power	Power input (ext) / Power output (USB)

4.3 Pin Map — Right Side (Pin 25–48)

DIP Pin	Signal Name	FPGA Pin	Direction	Function
25	**GND**	—	Power	Ground reference
26	GPIO_D6	R3	I/O	General purpose GPIO, Group D bit 6
27	GPIO_D5	T3	I/O	General purpose GPIO, Group D bit 5
28	GPIO_D4	R2	I/O	General purpose GPIO, Group D bit 4
29	GPIO_D3	T1	I/O	General purpose GPIO, Group D bit 3
30	GPIO_D2	T2	I/O	General purpose GPIO, Group D bit 2
31	GPIO_D1	U1	I/O	General purpose GPIO, Group D bit 1
32	GPIO_D0	W2	I/O	General purpose GPIO, Group D bit 0

DIP Pin	Signal Name	FPGA Pin	Direction	Function
33	INTR_3	V2	Input	External interrupt input 3
34	PWM_1	W3	Output	PWM output channel 1
35	SPI_SCLK	V3	Output	SPI clock (software-settable, up to 25 MHz)
36	SPI_MOSI	W5	Output	SPI master-out slave-in
37	SPI_MISO	V4	Input	SPI master-in slave-out
38	SPI_SS0	U4	Output	SPI slave select 0 (active low)
39	SPI_SS1	V5	Output	SPI slave select 1 (active low)
40	PWM_2	W4	Output	PWM output channel 2
41	INTR_2	U5	Input	External interrupt input 2
42	GPIO_C6	U2	I/O	General purpose GPIO, Group C bit 6
43	GPIO_C5	W6	I/O	General purpose GPIO, Group C bit 5
44	GPIO_C4	U3	I/O	General purpose GPIO, Group C bit 4
45	GPIO_C3	U7	I/O	General purpose GPIO, Group C bit 3
46	GPIO_C2	W7	I/O	General purpose GPIO, Group C bit 2
47	GPIO_C1	U8	I/O	General purpose GPIO, Group C bit 1
48	GPIO_C0	V8	I/O	General purpose GPIO, Group C bit 0

4.4 On-Board I/O (No DIP Pin Exposure)

Function	FPGA Pins
LEDs (2)	A17, C16
User button (BTN1)	B18
Reset button (BTN0)	A18
RGB LED	B17, B16, C17

5 Peripherals

All peripheral registers are memory-mapped and reached through the AXI data port; base addresses follow the class-based scheme summarised in Section 2.7. Pin locations for every signal are listed in the pin maps of Section 4. The QSPI flash controller is described with the memory system in Section 3.5.

5.1 GPIO

All GPIO groups are memory-mapped ports with per-bit direction control: the TRI register sets each pin's direction, the DATA register reads/writes the pin. Vitis driver: `XGpio`.

5.1.1 On-Board GPIO

Instance	Base Address	Width	Direction	Connection	Description
`board_led_2bits`	`0x4000_0000`	2	Output	A17, C16	On-board LEDs × 2
`board_button`	`0x4001_0000`	1	Input	B18	On-board user button (BTN1)
`board_rgb`	`0x4002_0000`	3	Output	B17, B16, C17	On-board RGB LED (R/G/B)

5.1.2 DIP Connector GPIO (4 Groups × 7-bit)

Instance	Base Address	Width	DIP Pins	Description
`gpio_A_0_6`	`0x4003_0000`	7	Pin 1–7	GPIO Group A, bidirectional I/O
`gpio_B_0_6`	`0x4004_0000`	7	Pin 17–23	GPIO Group B, bidirectional I/O
`gpio_C_0_6`	`0x4005_0000`	7	Pin 42–48	GPIO Group C, bidirectional I/O
`gpio_D_0_6`	`0x4006_0000`	7	Pin 26–32	GPIO Group D, bidirectional I/O

Description: Each group is a 7-bit bidirectional port; every bit can be an input or an output independently.

5.1.3 External Interrupt Inputs (`INT_0_3`)

Item	Value
Base Address	`0x4007_0000`
Width	4-bit, input-only
DIP Pins	Pin 8 (INTR_0), Pin 9 (INTR_1), Pin 41 (INTR_2), Pin 33 (INTR_3)
Interrupt	INTC In5

Description: Four external interrupt inputs grouped into a single GPIO instance with interrupt generation enabled — a change on any of the four pins can raise INTC In5.

5.2 Timers and PWM

Six 32-bit AXI timer instances (Vitis driver: `XTmrCtr`) — three as general-purpose timers with interrupts, three with their outputs routed to pins as PWM channels.

5.2.1 System Timers

Instance	Base Address	Interrupt	Description
`timer_0`	`0x4010_0000`	INTC In0	General-purpose system timer
`timer_1`	`0x4011_0000`	INTC In1	General-purpose timer
`timer_2`	`0x4012_0000`	INTC In2	General-purpose timer

5.2.2 PWM Outputs

Instance	Base Address	Output Pin	DIP Pin	Description
`PWM_0`	`0x4020_0000`	J3	Pin 10	PWM Channel 0
`PWM_1`	`0x4021_0000`	W3	Pin 34	PWM Channel 1
`PWM_2`	`0x4022_0000`	W4	Pin 40	PWM Channel 2

Description: Timer instances configured in PWM mode to generate square-wave outputs. Useful for LED dimming, motor speed control, buzzer tone generation, etc. Frequency and duty cycle are configured through the Timer Load Registers and the PWM enable bit.

5.3 UART

5.3.1 USB UART (`uart_USB`)

Item	Value
Base Address	`0x4030_0000`
TX / RX Pins	J18 / J17 — routed to the on-board Micro-USB connector
Interrupt	INTC In4

Description: 16550-compatible UART for host PC communication through the on-board Micro-USB connector. Baud rate is software-configured via the divisor latch: at 100 MHz system clock, divisor 54 (0x36) gives 115200 baud. Typically mapped as STDIN/STDOUT.

5.3.2 External UART (`uart_1`)

Item	Value
Base Address	`0x4031_0000`
TX / RX Pins	J1 (DIP 11) / K2 (DIP 12)
Interrupt	INTC In3

Description: Second 16550 UART exposed on the DIP connector for communication with external devices (e.g., sensor modules, Bluetooth modules).

5.4 I2C Controller

Item	Value
Base Address	`0x4070_0000`
SCL Frequency	100 kHz (standard mode)
Input Glitch Filter	50 ns on SCL and SDA — per the I2C tSP spike-suppression spec
SCL / SDA Pins	DIP Pin 13 (L1) / Pin 14 (L2)
Pull-ups	Weak FPGA internal pull-ups enabled; **external 4.7 kΩ to 3.3 V recommended** for real devices
Interrupt	INTC In6

Description: AXI IIC master for external I2C devices (sensors, EEPROMs, OLED displays). Open-drain signaling: any device may only pull the line low; the pull-up resistor returns it high. Devices are addressed by their 7-bit I2C address. Interrupt routing is listed in Section 2.3. Use the Vitis `XIic` driver.

5.5 SPI Master

Item	Value
Base Address	`0x4080_0000`
SCLK	Software-programmable, ≈1.6 – 25 MHz (**6.25 MHz at reset**)
Clock Control	`0x4090_0000` — runtime clock setting; see `showcase/src/spi0.c` for the `spi0_set_clock()` helper
Slave Selects	2 — two devices can share the bus
Pins	DIP 35 SCLK (V3) · 36 MOSI (W5) · 37 MISO (V4) · 38 SS0 (U4) · 39 SS1 (V5)
Interrupt	INTC In7

Description: SPI master for external devices (displays, ADCs, flash modules). Full-duplex push-pull signaling — no pull-ups needed; the active device is chosen by driving its SS line low. Use the Vitis `XSpi` driver. The serial clock is set at runtime through the clock-control block: slow it down for breadboard wiring or long cables, speed it up for short-wired display modules (`spi0_set_clock(hz)` — one call, takes effect immediately).

> **Note:** This is a *second, independent* SPI controller — do not confuse it with `axi_quad_spi_0` (`0x4050_0000`), which is dedicated to the on-board QSPI boot flash. Firmware should select controllers by instance macro (`XPAR_SPI_0_BASEADDR` vs `XPAR_AXI_QUAD_SPI_0_BASEADDR`), never by generic `XPAR_XSPI_n_*` numbering.

5.6 Analog-to-Digital Converter (XADC)

Item	Value
Base Address	`0x4060_0000`

Item	Value
Resolution	12-bit
Conversion Rate	500 KSPS aggregate
Sequencer	Continuous scan over the 5 enabled channels

Enabled Channels:

Channel	Source	Description
VAUX4	DIP Pin 15 (G3/G2)	External analog input 0 (on-board divider: 0–3.3 V → 0–1 V)
VAUX12	DIP Pin 16 (H2/J2)	External analog input 1 (on-board divider: 0–3.3 V → 0–1 V)
Temperature	Internal	FPGA die temperature monitor
VCCINT	Internal	Core voltage monitor (1.0 V)
VCCAUX	Internal	Auxiliary voltage monitor (1.8 V)

Description: The Artix-7 built-in 12-bit ADC capable of measuring external analog signals and monitoring internal FPGA temperature and supply voltages. External input pins pass through an on-board resistive voltage divider (2.32 KΩ / 1 KΩ, ratio ≈ 0.301), accepting up to 3.3 V at the DIP pin.

Effective Per-Channel Sampling Rate: The sequencer continuously cycles through all 5 enabled channels (VAUX4, VAUX12, Temperature, VCCINT, VCCAUX). The aggregate conversion rate is 500 KSPS, giving each channel an effective rate of 500 K ÷ 5 = **100 KSPS**.

5.6.1 Analog Input Circuit

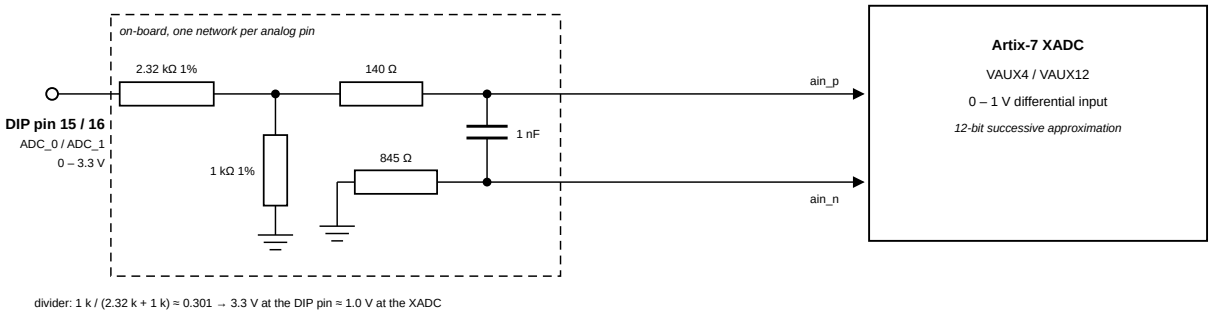

Figure 6: On-board voltage divider circuit for XADC analog inputs

The XADC expects an input range of 0–1 V. The board includes a resistive voltage divider (2.32 KΩ / 1 KΩ, all 1% precision) that scales the DIP pin voltage down to the FPGA's acceptable range. A 140 Ω series resistor and 1 nF capacitor are placed on the differential pair for filtering. The ADx_N path has an 845 Ω series resistor.

Parameter	Value
Max input voltage on DIP pin 15/16	**3.3 V** (relative to GND on pin 25)
Voltage at FPGA after divider	0–1 V
Divider ratio	1 KΩ / (2.32 KΩ + 1 KΩ) ≈ 0.301
Resistor tolerance	1%

> **Note:** Pins 15 and 16 are routed through on-board voltage dividers. If used as analog inputs, they must **not** be assigned as digital I/O in the constraints file. Do not exceed 3.3 V on these pins.

6 Electrical Characteristics and Power

6.1 Electrical Characteristics

Parameter	Value
I/O Standard	LVCMOS33 (3.3 V)
Series Resistance on DIP Pins	None
Max Input Voltage (digital I/O)	abs. max –0.4 V to VCCO + 0.55 V = **3.85 V** (DS181); not 5 V tolerant
Analog Input Range (Pin 15, 16)	0–3.3 V (on-board divider scales to the XADC's 0–1 V; see Section 5.6.1)
Clock Source	12 MHz oscillator (FPGA pin L17), PLL → 100 MHz

- When a USB host is attached, the VU pin is driven to USB voltage (4.5–5.5 V). Any external power source on VU **must be disconnected first** before plugging in USB, or the external supply may be damaged.
- This is **especially dangerous with battery power sources**, as batteries cannot tolerate reverse current.
- The FPGA I/O pins on the DIP connector have **no series resistors**. The Artix-7 absolute maximum input rating is VCCO + 0.55 V = **3.85 V** (and –0.4 V minimum, per DS181) — the pins are **not 5 V tolerant**; use a level shifter or divider for 5 V devices.

6.2 Power Architecture

The Cmod A7-35T uses a **Linear Technologies LTC3569** triple-output buck regulator (IC10) to generate all onboard supply voltages from a single input rail called **VU**.

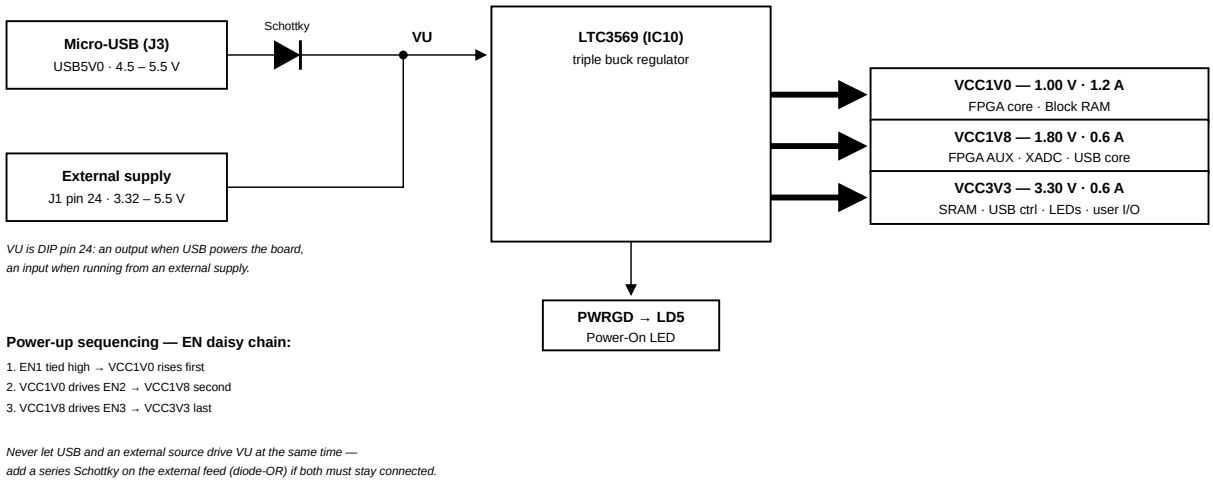

Figure 7: Cmod A7 power supply tree

6.3 Power Input Options

The board can be powered from either of the following sources:

Source	Connector	Schematic Net	Min Voltage	Recommended	Max Voltage
USB	J3 (Micro-USB)	USB5V0	4.5 V	5.0 V	5.5 V
External Supply	J1 Pin 24 (DIP)	VU	3.32 V*	5.0 V	5.5 V

> **Note:** * The minimum external voltage on VU depends on the current drawn from the VCC3V3 rail via the Pmod header:
>
> - 0 mA drawn → minimum 3.32 V
> - 100 mA drawn → minimum 3.38 V
> - 250 mA drawn → minimum 3.48 V

GND reference is on **J1 Pin 25**.

6.4 Output Power Rails

All three rails are generated by the LTC3569 from the VU input.

Rail	Typical Voltage	Min / Max Voltage	Max Current	Powered Devices
VCC1V0	1.00 V	0.95 V / 1.05 V	1.2 A	FPGA Core, Block RAM
VCC1V8	1.80 V	1.71 V / 1.89 V	0.6 A	FPGA AUX, ADC, USB Core
VCC3V3	3.30 V	3.135 V / 3.465 V	0.6 A	SRAM, USB Controller, Pmod, LEDs, Buttons, FPGA User I/O

6.4.1 Power-Up Sequence

The LTC3569 enable pins are daisy-chained to enforce a sequential startup:

1. **VCC1V0** starts first (EN1 tied high)
2. **VCC1V8** starts after VCC1V0 is stable (EN2 driven by VCC1V0)
3. **VCC3V3** starts after VCC1V8 is stable (EN3 driven by VCC1V8)

The **PWRGD** (Power Good) output drives the Power On LED (LD5).

6.5 VU Pin Behavior

Condition	VU Pin Function
USB connected	VU is an **output** — driven by USB 5V through a Schottky diode (expect ~5 V with small drop)
USB not connected	VU is an **input** — accepts external power supply (3.32–5.5 V)

6.6 Dual Power Source (USB + External)

The on-board Schottky diode is located on the **USB side** (between USB VBUS and VU), not on the VU input pin. This means:

Scenario	Safe?	Notes
USB only	✓	Normal operation
External only (VU pin)	✓	Normal operation
Both connected, only one active	✓	The inactive source supplies no current, no conflict
Both connected **and both supplying power**	✗	Two voltage sources fight on the same VU node
Both connected and both active, **with external Schottky added**	✓	Higher-voltage source wins; lower is blocked

Key distinction: Physically connecting both cables is not the problem — the danger is having **both sources actively supplying voltage at the same time** without isolation.

If you need to have both connected simultaneously with both powered (e.g., USB for JTAG/UART while also on external supply), **add a Schottky diode in series with the external supply on the VU pin**. This creates a “diode-OR” circuit where whichever source has the higher voltage automatically takes over, and the other is safely blocked.

6.7 Quick Reference for External Power Design

Parameter	Value
Input Connector	J1 Pin 24 (VU) + Pin 25 (GND)
Recommended Input Voltage	5.0 V
Acceptable Input Range	3.32 – 5.5 V
Total Max Current (all rails)	~2.4 A (1.2 + 0.6 + 0.6)
Power Good Indicator	LD5 (LED)
Regulator IC	LTC3569
Protection Required for Dual Supply	Schottky diode in series with VU

7 Boot and Deployment

This chapter describes how firmware is stored on the board and started at power-on. The application resides in the QSPI flash. A small bootloader (AMD/Xilinx's standard `srec_spi_bootloader`, part of the hardware image) copies it into SRAM and starts it, with no PC attached, in under a second after power is applied.

7.1 Boot Modes: JTAG vs Standalone

Both modes coexist on the same board with no switching:

	JTAG (Vitis Run/Debug)	Standalone (flash)
Code goes to	RAM directly (volatile)	Flash (persistent)
After power-cycle	back to the flashed app	your app boots
Breakpoints / stepping	yes	no
Typical use	daily development loop	shipping a finished program

The everyday loop is edit, build, JTAG Run/Debug. When the program works, deploy it to flash once. JTAG always takes priority over the bootloader: it halts the CPU before the bootloader runs, so the flash contents cannot interfere with a debug session.

7.2 How Standalone Boot Works

```
Power-on ──► FPGA configures itself from QSPI flash (<1 s)
        ──► Bootloader (in Block RAM, part of the hardware image) starts
        ──► Reads the application image (SREC) from flash @ 0x220000
        ──► Copies it to the addresses the image names (SRAM)
        ──► Jumps to the application's entry point
```

The application is stored as an SREC image (Motorola S-record), a text format in which every line carries its own destination address. The bootloader copies each record to the address it names, so the linker script decides where the program runs. The course template links to SRAM at `0x6000_0000`, and that is where the image ends up.

Memory roles:

Memory	Size	Role
QSPI flash	4 MB	hardware image + bootloader (first 2.2 MB) · application slot @ `0x220000` (non-volatile)
SRAM @ `0x60000000`	512 KB	where your application executes (code/data/heap, I/D-cached)
BRAM @ `0x00000000`	128 KB	32 KB bootloader (untouchable) · 32 KB ITCM @ `0x8000` (your fast code) · 64 KB DTCM @ `0x10000` (stack + fast data)

> **Note:** The bootloader is part of the hardware image and is restored from flash at every power-on. Like a mask ROM, it cannot be corrupted from software.

7.3 Build Your Application

Create a Vitis application as in the separate JTAG Debug Mode guide (sections 1–7), using the course template `workspace-example/app_template/src/` — its `main.c` and `lscript.ld` replace the generated Hello World sources.

The template's linker script places:

- `.text` / `.rodata` / `.data` / `.bss` / heap → **SRAM** (512 KB, cached)
- stack → top of **DTCM** (64 KB BRAM @ `0x10000`; 1-cycle, never collides with the bootloader)
- functions tagged `ITCM_FUNC` → **ITCM** (32 KB BRAM @ `0x8000`); variables tagged `DTCM_DATA` → DTCM

ITCM code ships inside the SRAM image and is copied out by `mcu_init()` at the top of `main()`, the same startup-copy idiom an STM32H7 uses for its ITCM. This is why the template has one rule: `mcu_init();` stays the first line of `main()`. (It also sets the UART to 115200.) Put interrupt handlers and timing-critical loops in ITCM — TCM never misses, so worst-case latency equals best-case.

The template prints a memory tour at boot, which shows the layout directly:

```
RISC-V MCU on Cmod A7 - memory tour
  main()       @ 0x600009D0   (SRAM,  cached)
  blink_step() @ 0x00008000   (ITCM,  1-cycle)
  blink_count  @ 0x00010000   (DTCM,  1-cycle)
  stack        @ 0x0001FFC0   (DTCM,  grows down)
```

> **Note:** The same ELF works unchanged in both modes: Vitis Run/Debug over JTAG during development, flash deployment to ship it. No rebuild is needed.

7.4 Deploy to Flash

Deployment = converting the built ELF to SREC and writing it into the flash application slot at `0x220000`:

1. **Convert to SREC** with the toolchain's `objcopy` (`riscv32-amd-linux-gnu-objcopy -O srec app.elf app.srec`). The record addresses come from the linker script — with the course template they all fall in SRAM.
2. **Write the SREC file's bytes to flash offset `0x220000`** over JTAG. The erase and write are scoped to the application slot, so the hardware image at the start of flash is never touched.
3. **Power-cycle (or reboot from flash)** — the bootloader picks up the new image.

(Step-by-step GUI walkthrough with screenshots: to be added.)

The slot runs from the end of the bitstream region to the end of the flash: 1.875 MB, more SREC text than a full-SRAM application converts to (SREC is about 3× the binary size). In practice the size limit is the 512 KB SRAM. Only code and initialized data count toward it; zero-initialized data and the heap take no space in the image.

7.5 Boot and Verify

- **Power-cycle** → the board boots your flashed app automatically. No PC needed.
- `xil_printf` (the SDK's lightweight `printf`) output arrives on the same USB cable at **115200 baud**. To watch it, open any serial terminal, e.g. `python3 -m serial.tools.miniterm /dev/ttyUSB1 115200` (ships with pyserial; quit with Ctrl-]). The board shows up as two ports — the UART is usually the higher-numbered one.
- A useful convention: light an LED near the top of `main()` so a running board is recognizable at a glance. The template's blinking LEDs serve this purpose.

> **Note:** The reset button (BTN0) restarts the CPU into the bootloader (BRAM is intact), which re-copies your app from flash, so in this design a reset behaves like a power-cycle. Modified `.data` globals are restored because the image is re-read from flash each boot.

7.6 One-Time Board Setup (Instructor)

Program the deployment image `release/official_boot.mcs` (hardware image + bootloader + preloaded demo application) into the QSPI flash:

1. Open Vivado **Hardware Manager > Open Target > Auto Connect**.
2. Right-click `xc7a35t_0` > **Add Configuration Memory Device** — pick the part matching the IC3 chip: `mx25l3273f-spi-x1_x2_x4` (Vivado's catalog entry covering the board's Macronix MX25L3233F — verified working) or `n25q32-3.3v-spi-x1_x2_x4` (Micron, older boards).
3. Right-click the memory device > **Program Configuration Memory Device**, select `release/official_boot.mcs`, keep **Erase / Program / Verify** checked, **OK**.
4. Power-cycle. The preloaded demo application starts (its banner appears on the USB serial console at 115200 baud).

Students never repeat this step; from here on the board is a pure MCU.

7.7 How the Deployment Image Is Made (Instructor Reference)

`release/official_boot.mcs` = the hardware image with the bootloader merged into its BRAM initialization, plus the demo application's SREC at `0x220000`. The full mechanics (BRAM init frames, `updatemem`, `write_cfgmem`, flash layout) are in the Boot Image Pipeline guide.

The bootloader is not custom code. It is Vitis's stock "SREC SPI Bootloader" application template, built with one configuration change in `src/blconfig.h`:

```
#define FLASH_IMAGE_BASEADDR  0x00220000
```

Board-verified facts for anyone rebuilding it:

1. **The flash controller is auto-selected correctly.** Vitis 2025.2 (System Device Tree flow) picks `XPAR_AXI_QUAD_SPI_0_BASEADDR`, which is this board's flash controller. No manual fix is needed.
2. **Keep `VERBOSE` off.** The template's debug prints assume an initialized UART divisor; on this board they hang before the jump (the CPU parks in `XUartNs550_SendByte`). It is undefined by default; leave it that way, or call `XUartNs550_SetBaud(...)` first.
3. The bootloader links to BRAM (`0x0`), so it can copy the application into SRAM without overwriting itself. Merging its ELF into the bitstream uses `updatemem` (about 6 seconds, no re-implementation); see the Boot Image Pipeline guide.
4. Applications must set their own UART baud before printing. The course template's `mcu_init()` does this.

7.8 Troubleshooting

1. **Garbage on the terminal** — Baud must be 115200; the app sets it via `mcu_init()`. The stock Hello World template does not set it; add `XUartNs550_SetBaud(XPAR_UART_USB_BASEADDR, 100000000, 115200);` or use the course template.
2. **Deployed, but silent after power-on** — The ELF was probably not linked with the course template's `lscript.ld`: the bootloader honors the SREC record addresses, and a default all-BRAM layout collides with the bootloader itself. Rebuild with the template and redeploy.
3. **No memory tour / garbage addresses** — `mcu_init()` is not the first line of `main()`, or it was removed: nothing tagged `ITCM_FUNC`/`DTCM_DATA` works before it runs.
4. **Board boots an old app** — the flash write did not complete (check the programming step's verify result), or you power-cycled before it finished. Redeploy.
5. **Nothing at all (no LED, no output)** — the FPGA did not configure, which means the flash image is invalid; redo Section 7.6. JTAG still works for recovery.
6. **Want a factory-blank board** — Hardware Manager > right-click the memory device > **Erase**. The FPGA then waits for JTAG at power-on.